



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección: 1

Tema 4 - Análisis léxico

Profesor:

Felix Marquez

Alumnos:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852

Puerto Ordaz, 12 de julio de 2026.

Introducción

Este informe desarrolla la actividad de Análisis Léxico de la asignatura Lenguaje y Compiladores, que aplica los conceptos de gramáticas de libre contexto y autómatas a la primera fase de todo compilador: el análisis léxico, es decir, el proceso de descomponer un programa fuente en una secuencia de componentes léxicos (tokens) a partir de sus lexemas.

El trabajo se organiza en cuatro partes complementarias. La primera aborda el autómata de pila como extensión del autómata finito, presentando su definición formal, ejemplos concretos de funcionamiento y una discusión sobre su poder computacional frente a los Autómatas Finitos Deterministas (AFD) y No Deterministas (AFND). La segunda traslada esa base teórica a la práctica mediante la construcción, desde cero y con expresiones regulares, de un analizador léxico para verificar archivos de configuración tipo Docker. La tercera introduce el uso de un metacompilador (Flex) para generar automáticamente un analizador léxico sobre un lenguaje L definido como subconjunto de Rust (MiniRust), permitiendo contrastar la construcción manual frente a la generación automática. La cuarta cierra con una reflexión aplicada sobre el papel de herramientas como Flex dentro de la seguridad informática.

En conjunto, estas cuatro actividades buscan demostrar tanto el dominio teórico de los autómatas como máquinas reconocedoras de lenguajes, como la capacidad práctica de traducir esa teoría en herramientas de software funcionales.

Autómatas de Pila

Un Autómata de Pila es una máquina teórica abstracta que extiende las capacidades de un autómata finito convencional mediante la adición de una memoria auxiliar estructurada como pila. Esta memoria de almacenamiento lineal ilimitado le permite recordar información arbitrariamente profunda, haciéndolo capaz de reconocer Gramáticas Libres de Contexto (GLC), las cuales se sitúan en el Tipo 2 de la Jerarquía de Chomsky.

Definición formal

Un Autómata a Pila se define como la séptupla: $(\Sigma, P, Q, A_0, q_0, f, F)$ donde:

- Σ : alfabeto de entrada.
- P : alfabeto de la pila.
- Q : conjunto de estados.
- A_0 : símbolo inicial de la pila (#).
- q_0 : símbolo inicial del conjunto de estados.
- f : función de transición. Es una aplicación de $Q \times \Sigma \cup \{\lambda\} \times P$ en el conjunto de partes de $P(Q \times P)^*$.
- F : conjunto de estados finales o de aceptación.

Ejemplo: El lenguaje clásico no regular

Este lenguaje requiere emparejar un número exacto de letras 'a' con el mismo número de letras 'b'. Un autómata finito no puede procesarlo al carecer de memoria para "contar", pero un AP lo resuelve fácilmente:

- **Lógica de operación:** Cada vez que el autómata lee una 'a', la introduce en la pila. Al momento de leer la primera 'b', el autómata cambia de estado y empieza a extraer (*pop*) una 'a' de la pila por cada 'b' leída en la entrada. Si la cadena termina justo cuando la pila vuelve a tener únicamente el símbolo de fondo Z_0 , la cadena es aceptada.

Tipos de Autómatas Finitos

Autómata Finito Determinista (AFD)

Es el tipo más estricto y eficiente en ejecución, para cada estado y para cada símbolo del alfabeto, existe exactamente una única transición posible. No hay ambigüedad; el camino a seguir está completamente determinado. Es extremadamente rápido de ejecutar en software mediante tablas de transición o estructuras switch-case.

Autómata Finito No Determinista (AFND)

Es un modelo más flexible y fácil de diseñar a mano, desde un estado, un mismo símbolo de entrada puede llevar a múltiples estados diferentes (o a ninguno). El autómata "explora" todos los caminos posibles en paralelo y acepta la cadena si al menos uno de ellos llega a un estado de aceptación. Simplifica enormemente el diseño de sistemas complejos antes de ser traducidos a código ejecutable.

Utilidades del Autómata de Pila

En el ámbito del software y la ingeniería informática, los autómatas de pila representan el motor computacional de la fase de **Análisis Sintáctico (Parser)** de un compilador. Mientras que la tokenización y el análisis léxico se resuelven eficientemente mediante expresiones regulares y Autómatas Finitos Determinísticos, las estructuras anidadas de los lenguajes de programación modernos requieren mayor poder de cómputo.

Sus utilidades principales incluyen:

- **Validación Sintáctica:** Análisis de la jerarquía gramatical de códigos fuente.
- **Evaluación de Expresiones:** Procesamiento de fórmulas aritméticas y lógicas respetando la precedencia de operadores mediante estructuras de tipo árbol sintáctico.

- **Procesamiento de Formatos Estructurados:** Análisis y parseo de archivos JSON, XML o HTML que se basan inherentemente en etiquetas y llaves de apertura y cierre balanceadas.

Importancia y Poder Computacional en Relación a los AFD y AFND

Característica	Autómatas Finitos (AFD / AFND)	Autómatas de Pila (AP)
Capacidad de Memoria	Finita y limitada únicamente a sus estados (Q)	Infinita (gracias a la estructura dinámica de la Pila)
Tipo de Lenguaje	Lenguajes Regulares (Tipo 3)	Lenguajes Libres de Contexto (Tipo 2)
Equivalencia Determinista	AFD = AFND (Tienen el mismo poder de reconocimiento)	AP Determinista $\subset$ AP no Determinista (El caso no determinista es estrictamente más poderoso)

La Limitación de los AFD y AFND

Tanto los AFD como los AFND procesan cadenas basándose estrictamente en su estado actual y el símbolo que leen. Como su número de estados es estrictamente finito, carecen de la capacidad de contar o recordar eventos pasados a gran escala. No pueden reconocer lenguajes simétricos o balanceados (como $a^n b^n$) ya que no tienen dónde almacenar el valor de n .

Al incorporar la pila, el AP trasciende la limitación de la memoria finita. No requiere almacenar información en la configuración de sus estados, sino que la "escribe" y "saca" de la pila. Es crucial acotar que en los AP, el No Determinismo aporta mayor poder de

reconocimiento. Los AP No Deterministas (APND) cubren la totalidad de las Gramáticas Libres de Contexto (GLC), mientras que los AP Deterministas (APD) solo reconocen un subconjunto (los lenguajes libres de contexto deterministas), los cuales, afortunadamente, bastan para diseñar la sintaxis formal de la gran mayoría de los lenguajes de programación del mundo real.

Lexer para Archivos Docker

Para la validación léxica de un archivo de configuración de contenedores ([Dockerfile](#)), se diseñó e implementó un analizador léxico (lexer) desde cero utilizando el lenguaje de programación Python. El diseño se fundamenta en la teoría de autómatas finitos y en el uso estratégico de expresiones regulares (regex) para identificar los componentes léxicos del lenguaje.

A continuación, se detalla el proceso de construcción estructurado en tres fases:

1. Definición de Componentes Léxicos (Tokens y Patrones):

Se identificaron las instrucciones principales de la sintaxis de Docker (tales como [FROM](#), [RUN](#), [COPY](#), [ENV](#), [WORKDIR](#), [EXPOSE](#), [CMD](#), [ENTRYPOINT](#), [VOLUME](#), [USER](#) y [ARG](#)). Para evitar solapamientos con palabras que contengan estas subcadenas, se utilizaron límites de palabra ([\b](#)) en las expresiones regulares correspondientes. Adicionalmente, se construyeron expresiones específicas para capturar comentarios (iniciados por <#>), cadenas de texto delimitadas por comillas ([STRING](#)), delimitadores ([ASSIGN](#), [BACKSLASH](#)), y argumentos o rutas generales ([ARGUMENT](#)).

2. Mapeo y Compilación en un Único Patrón (Mapeo AFD):

Siguiendo el enfoque clásico del procesamiento léxico eficiente, se unificaron todas las expresiones regulares en un único patrón maestro utilizando grupos nombrados bajo la nomenclatura ([?P<nombre_token>patrón](#)). Este patrón compilado es procesado por el motor de expresiones regulares de Python a través de [re.finditer](#), simulando el comportamiento de un Autómata Finito Determinista (AFD) que realiza la lectura carácter por carácter a lo largo del flujo de entrada.

3. Estructura de Control de Errores y Ubicación Geográfica:

El analizador realiza un seguimiento activo del número de líneas y columnas. Cada salto

de línea (`\n`) incrementa el contador y actualiza el índice de inicio de línea para calcular con precisión la columna de los tokens subsiguientes. Finalmente, para garantizar la robustez, se definió una regla de salvaguarda denominada **MISMATCH** al final de la prioridad léxica. Si el puntero lee un carácter no contemplado por las reglas sintácticas de Docker, el lexer interrumpe inmediatamente la ejecución arrojando un error detallado con la coordenada exacta del fallo léxico.

Ejemplos de ejecución (Casos de prueba).

Caso de Prueba 1: Ejecución de un Dockerfile estándar correcto

Archivo de entrada (**Dockerfile_correcto**):

```
# Imagen base de Python

FROM python:3.9-slim

WORKDIR /app

COPY . /app

RUN pip install -r requirements.txt
```

Salida obtenida en consola:

```
--- Analizando: Dockerfile_correcto ---
```

```
Token: COMMENT    | Lexema: # Imagen base de Python | Línea: 1 | Columna: 0
Token: NEWLINE    | Lexema: \n                | Línea: 1 | Columna: -27
Token: FROM       | Lexema: FROM              | Línea: 2 | Columna: 0
Token: ARGUMENT   | Lexema: python:3.9-slim   | Línea: 2 | Columna: 5
Token: NEWLINE    | Lexema: \n                | Línea: 2 | Columna: -21
Token: WORKDIR    | Lexema: WORKDIR           | Línea: 3 | Columna: 0
Token: ARGUMENT   | Lexema: /app              | Línea: 3 | Columna: 8
Token: NEWLINE    | Lexema: \n                | Línea: 3 | Columna: -13
```

Token: COPY | Lexema: COPY | Línea: 4 | Columna: 0
 Token: ARGUMENT | Lexema: . | Línea: 4 | Columna: 5
 Token: ARGUMENT | Lexema: /app | Línea: 4 | Columna: 7
 Token: NEWLINE | Lexema: \n | Línea: 4 | Columna: -12
 Token: RUN | Lexema: RUN | Línea: 5 | Columna: 0
 Token: ARGUMENT | Lexema: pip | Línea: 5 | Columna: 4
 Token: ARGUMENT | Lexema: install | Línea: 5 | Columna: 8
 Token: ARGUMENT | Lexema: -r | Línea: 5 | Columna: 16
 Token: ARGUMENT | Lexema: requirements.txt | Línea: 5 | Columna: 19

Caso de Prueba 2: Definición de variables y puertos de red

Archivo de entrada (**Dockerfile_variables**):

```
FROM alpine
ENV PORT=8080
EXPOSE 8080
```

Salida obtenida en consola:

--- Analizando: Dockerfile_variables ---

Token: FROM | Lexema: FROM | Línea: 1 | Columna: 0
 Token: ARGUMENT | Lexema: alpine | Línea: 1 | Columna: 5
 Token: NEWLINE | Lexema: \n | Línea: 1 | Columna: -12
 Token: ENV | Lexema: ENV | Línea: 2 | Columna: 0
 Token: ARGUMENT | Lexema: PORT | Línea: 2 | Columna: 4
 Token: ASSIGN | Lexema: = | Línea: 2 | Columna: 8
 Token: ARGUMENT | Lexema: 8080 | Línea: 2 | Columna: 9
 Token: NEWLINE | Lexema: \n | Línea: 2 | Columna: -14

Token: EXPOSE | Lexema: EXPOSE | Línea: 3 | Columna: 0

Token: ARGUMENT | Lexema: 8080 | Línea: 3 | Columna: 7

Caso de Prueba 3: Detección y control de errores léxicos (MISMATCH)

Archivo de entrada (Dockerfile_error):

FROM ubuntu

RUN apt-get update %

Salida obtenida en consola:

--- Analizando: Dockerfile_error ---

Token: FROM | Lexema: FROM | Línea: 1 | Columna: 0

Token: ARGUMENT | Lexema: ubuntu | Línea: 1 | Columna: 5

Token: NEWLINE | Lexema: \n | Línea: 1 | Columna: -12

Token: RUN | Lexema: RUN | Línea: 2 | Columna: 0

Token: ARGUMENT | Lexema: apt-get | Línea: 2 | Columna: 4

Token: ARGUMENT | Lexema: update | Línea: 2 | Columna: 12

[ERROR DETECTADO] Error léxico: Carácter inesperado '%' en la línea 2, columna 19

Lexer para un Subconjunto de Rust

-Propósito

`L` (que llamaremos **MiniRust**) es un subconjunto del lenguaje de programación Rust. No implementa análisis sintáctico ni semántico: su único objetivo es definir con precisión qué cadenas de caracteres son válidas como *tokens* del lenguaje, para luego construir un lexer con un metacompilador.

-Alfabeto (Σ)

El alfabeto de entrada de `L` está compuesto por:

- Letras mayúsculas y minúsculas: `a-z`, `A-Z`
- Dígitos: `0-9`
- El guion bajo: `\_`
- Símbolos de operadores y puntuación: `+ - \* / % = < > ! & | ^ ( ) { } [ ] ; , : . "`
- Comillas dobles `""` (para strings) y comillas simples `` (para caracteres)
- Espacios en blanco, tabuladores y saltos de línea (separadores, no generan token)

- Categorías léxicas (tokens) de L

Palabras reservadas (keywords)

Categoría	Palabras reservadas
Declaración	`fn`, `let`, `mut`, `struct`, `enum`, `impl`, `const`, `static`
Control de flujo	`if`, `else`, `while`, `for`, `in`, `loop`, `break`, `continue`, `return`, `match`
Módulos / visibilidad	`pub`, `use`, `mod`
Referencias a sí mismo	`self`, `Self`

Conversión de tipos	`as`
---------------------	------

Literales booleanos

`true`, `false`

Tipos primitivos y de biblioteca (subconjunto soportado)

`i8 i16 i32 i64` (enteros con signo), `u8 u16 u32 u64` (enteros sin signo),

`f32 f64` (punto flotante), `bool`, `char`, `str`, `String`, `Vec`

Identificadores

Un identificador comienza con una letra o guion bajo, seguido de cero o más letras, dígitos o guiones bajos. Expresión regular:

...

IDENT = `[a-zA-Z_][a-zA-Z0-9_]*`

...

Ejemplos válidos: `contador`, `\_temp`, `valor1`, `Punto`.

Literales numéricos

- Entero: secuencia de dígitos, con guion bajo permitido como separador visual (igual que en Rust real, p. ej. `1\_000\_000`).

...

INTEGER = `[0-9]([0-9]|_)*`

...

- Flotante: entero, punto decimal, y parte fraccionaria obligatoria.

...

FLOAT = `[0-9]([0-9]|_)*\.[0-9]([0-9]|_)*`

...

Literales de texto y carácter

- String: cadena entre comillas dobles, admite secuencias de escape (`\"`, `\\n`, etc.) y no puede contener un salto de línea sin escapar.

...

```
STRING = "([^\\"n]|\\.)*"
```

...

- Carácter: exactamente un carácter (o una secuencia de escape) entre comillas simples.

...

```
CHARLIT = '([^\\"n]|\\.).'
```

...

Operadores

- Compuestos (deben reconocerse **antes** que los simples, por máxima longitud de coincidencia): `== != <= >= && || -> => :: += -= \*= /= %= << >>`
- Simples: `+ - \* / % = < > ! & | ^`

Delimitadores y puntuación

```
`(){}[];,:.`
```

Comentarios (se descartan, no producen token)

- Línea: `//` hasta el fin de línea.
- Bloque: `/\* ... \*/` (no anidado, igual que en Rust).

-Reglas de precedencia de reconocimiento

Flex aplica la regla de "coincidencia más larga" (*longest match*) y, ante empates,

la regla que aparece primero en el archivo ``l``. Por eso ``mini_rust.l`` respeta este orden:

1. Comentarios
2. Palabras reservadas y tipos (para que ``let`` no se reconozca como ``IDENT``)
3. Literales booleanos
4. Literales numéricos, de texto y de carácter
5. Operadores compuestos (para que ``==`` no se rompa en dos ``=``)
6. Operadores simples
7. Puntuación
8. Identificadores (captura todo lo que no cayó en las reglas anteriores)
9. Espacios en blanco
10. Cualquier otro carácter → **`**error léxico**`**

-Fuera del alcance de L

Para mantener acotado el ejercicio, quedan fuera de ``L``: genéricos (``<T>``), **lifetimes** (``'a``), macros (``println!``), atributos (``#[derive(...)]``), **closures** (``|x| x + 1``), operadores de rango (``..``, ``..=``) más allá de lo mínimo, y el sistema de módulos con rutas complejas. Estas extensiones podrían incorporarse en una iteración futura del lexer.

Aplicación en Seguridad Informática

1. La crisis de escalabilidad en el procesamiento de eventos

El paradigma actual de la seguridad informática no reside solo en la detección, sino en la latencia de respuesta. Un SIEM o un IDS de próxima generación (NG-IDS) no solo analiza tráfico básico; debe inspeccionar cargas útiles (payloads), normalizar logs de diversas fuentes y aplicar lógica de correlación compleja.

Al utilizar un metacompilador como FLEX, no estamos simplemente "generando código", estamos estandarizando el front-end de la ciberseguridad. Si el analizador léxico está optimizado mediante autómatas finitos deterministas (DFA), el costo de procesar cada byte de una trama de red es constante, independientemente del número de reglas. Esto es vital: si la complejidad del análisis fuera lineal respecto al número de reglas, el sistema colapsaría ante el crecimiento de las bases de datos de firmas de amenazas.

2. Metacompiladores como factor de resiliencia

La ciberseguridad exige agilidad. Ante una nueva vulnerabilidad (como un *Zero-Day*), el tiempo es el activo más valioso. La implementación de un lexer "hecho a mano" es propensa a errores (memory leaks, desbordamientos de buffer) y difícil de mantener. FLEX, por el contrario, abstrae la complejidad de la implementación de la tabla de transiciones del autómata, permitiendo que el equipo de seguridad se enfoque en la expresión del dominio (el lenguaje de la amenaza) y no en la ingeniería de bajo nivel.

3. Aplicación en otros lenguajes de seguridad

Más allá de las reglas de Snort, existen otros dominios críticos donde un metacompilador aportaría un valor inmenso:

- **Lenguajes de descripción de políticas de Firewall (ej. Iptables/Nftables):** La tokenización de reglas complejas que involucran sets de IPs, interfaces de red y marcado de paquetes.
- **Analizadores de logs (Formatos como CEF o Syslog):** Los dispositivos de red emiten mensajes en formatos variados. Un analizador léxico robusto es la primera línea de defensa para normalizar estos logs antes de ser inyectados en un motor de inteligencia artificial para detección de anomalías.
- **Lenguajes de marcado para firmas de Malware (ej. YARA):** YARA es, esencialmente, un DSL para clasificar malware. La capacidad de tokenizar estructuras complejas de archivos (headers, secciones, offsets) para buscar patrones binarios es un caso de uso donde la precisión de un metacompilador es superior a cualquier script simple.

Análisis de Caso: Tokenización de una Regla YARA

Para cumplir con el requisito de mostrar la tokenización, analicemos cómo se descompondría una regla de YARA, que es el estándar *de facto* para la búsqueda de patrones en archivos binarios.

Regla YARA de ejemplo:

condition: \$a at 0 and filesize < 100KB

Proceso de Tokenización (Lexer para YARA)

El metacompilador debe distinguir entre identificadores de variables, operadores lógicos, restricciones de posición y unidades de medida.

Lexema	Categoría (Token)	Explicación
--------	-------------------	-------------

condition	TK_CONDITION_KW	Palabra clave que inicia la sección de lógica.
:	TK_COLON	Delimitador estructural.
\$a	TK_VARIABLE	Referencia a un string definido previamente.
at	TK_OPERATOR_POS	Operador de posición absoluta.
0	TK_INTEGER	Valor numérico del offset.
and	TK_LOGIC_AND	Operador lógico de conjunción.
filesize	TK_PROPERTY	Propiedad intrínseca del archivo analizado.
<	TK_REL_OP	Operador relacional de comparación.
100KB	TK_SIZE_VALUE	Valor con unidad de medida (requiere conversión).

Implementación en FLEX (Fragmento)

C

%%

"condition" { return TK_CONDITION_KW; }

":" { return TK_COLON; }

\\\$[a-zA-Z0-9_]+ { return TK_VARIABLE; }

"at" { return TK_OPERATOR_POS; }

"and"|"or"|"not" { return TK_LOGIC; }

```
"filesize"      { return TK_PROPERTY; }  
[0-9]+          { return TK_INTEGER; }  
[0-9]+[KMG]B    { return TK_SIZE_VALUE; }  
"<"|">"|""=="  { return TK_REL_OP; }  
[ \t\n]+        { /* Espaciado */ }  
.  
%%
```

Conclusión

El desarrollo de las cuatro actividades permitió abordar el análisis léxico desde ángulos complementarios. En el plano teórico, el estudio del autómata de pila evidenció que, si bien los AFD/AFND son suficientes para reconocer los lenguajes regulares que conforman los tokens de un lenguaje de programación, existen estructuras —como el balanceo de paréntesis o construcciones anidadas— que exceden el poder de un AFD y requieren la memoria adicional de una pila. Esta distinción explica por qué el análisis léxico y el sintáctico se implementan como fases separadas de un compilador.

En el plano práctico, construir el lexer de Docker desde cero permitió experimentar de primera mano el proceso completo de tokenización, desde la definición de patrones hasta el manejo de errores léxicos. Por su parte, construir el lexer de MiniRust con Flex demostró el valor de las herramientas de metacompilación: en lugar de codificar manualmente el AFD correspondiente a cada patrón, Flex permitió declarar esos patrones como expresiones regulares y obtener directamente un analizador funcional en C, reduciendo el esfuerzo de implementación sin sacrificar control. La comparación entre ambos enfoques —manual y automatizado— deja claro que la elección depende del contexto: control total en un caso, rapidez y confiabilidad en el otro.

Finalmente, la reflexión sobre aplicaciones de Flex en seguridad informática conectó la teoría con usos reales, como el análisis de logs, la validación de configuraciones y la detección de patrones maliciosos.

En síntesis, el trabajo une la teoría de autómatas con la construcción de herramientas de software reales, sentando las bases necesarias para abordar más adelante la fase de análisis sintáctico dentro del proceso de compilación.

Referencias bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.

Docker Inc. (2026). *Dockerfile reference*. Docker Docs.
<https://docs.docker.com/engine/reference/builder/>

Márquez, F. (2026). *Tema 4 (Análisis Léxico): Autómatas finitos y expresiones regulares* [Guía de estudio]. Coordinación de Ingeniería en Informática, Universidad Nacional Experimental de Guayana.

Python Software Foundation. (2026). *re — Regular expression operations*. Python 3.12 Documentation. <https://docs.python.org/3/library/re.html>

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson/Addison-Wesley.

GeeksforGeeks. (2026). Flex (Fast Lexical Analyzer Generator). Recuperado el 14 de julio de 2026, de <https://www.geeksforgeeks.org/compiler-design/flex-fast-lexical-analyzer-generator/>

Klabnik, S., & Nichols, C. (2023). *The Rust programming language*. No Starch Press.
<https://doc.rust-lang.org/book/>

Paxson, V. (2016). *Lexical analysis with flex, for flex 2.6.2* [Manual de referencia].
<https://westes.github.io/flex/manual/>

Rust Compiler Team. (s.f.). Lexing and parsing — Rust Compiler Development Guide.
Recuperado el 14 de julio de 2026, de <https://rustc-dev-guide.rust-lang.org/the-parser.html>

Alvarado, M. (s.f.). *Lógica Computacional: Autómatas de pila e intérpretes*. Instituto de Investigaciones Filosóficas, Universidad Nacional Autónoma de México (UNAM).

Franco, E. A. (2018). *Clase 17: Autómatas de pila*. Teoría de la Computación.

StudySmarter. (2023). *Autómatas de Pila (Pushdown Automata): Definición, clasificación y balanceo de paréntesis*. StudySmarter Ciencias de la Computación.